

**Desarrollo de una Arquitectura Cliente-Servidor HTTP
para el Cálculo y Simulación de Ecuaciones Diferenciales
mediante Realidad Virtual**

Universidad Nacional de Cuyo

Facultad de Ingeniería

Realidad Virtual (00318)

Proyecto Final de Cátedra

Profesores

Prof. Rosenstein, Javier

Prof. Tomba, Carlos

Alumnos

Escobar, Matías – XXXXX

Martín Duci, Ignacio – 13560

Julio 2026

Índice

1. Introducción	4
2. Objetivos y Beneficiarios	5
2.1. Objetivo general	5
2.2. Objetivos específicos	5
2.3. Beneficiarios	5
3. Resolución de Ecuaciones Diferenciales	6
3.1. Fundamentos del método de diferencias finitas	6
3.2. Ecuación de onda 2D	6
3.2.1. Formulación matemática	6
3.2.2. Esquema leapfrog de segundo orden	6
3.2.3. Condición de estabilidad CFL	7
3.2.4. Condiciones iniciales implementadas	7
3.2.5. Implementación en Go	7
3.3. Ecuación de calor 2D	8
3.3.1. Formulación matemática	8
3.3.2. Esquema FTCS explícito	9
3.3.3. Condición de estabilidad	9
3.3.4. Condiciones iniciales y de contorno implementadas	9
3.3.5. Implementación en Go	10
3.4. Consideraciones de eficiencia	11
4. Arquitectura del Servidor	12
4.1. Resumen de rutas	12
4.2. <code>main.go</code> — Router y punto de entrada	14
4.3. <code>auth.go</code> — Autenticación y limitación de tasa	15
4.4. <code>sync.go</code> — Estado global en memoria	17
4.5. <code>handlers.go</code> — Endpoints de la app móvil	20
4.6. <code>admin.go</code> — Endpoints del panel docente	23
5. Trabajo Futuro	26
5.1. Escalabilidad de la plataforma	26
5.2. Incorporación de nuevos solvers	26
5.3. Mejoras en la experiencia de realidad virtual	27

Resumen

El presente trabajo describe el desarrollo de un sistema cliente-servidor orientado al entorno académico para el cálculo numérico y la visualización interactiva de ecuaciones diferenciales parciales (EDPs). El sistema fue concebido como una plataforma de exploración y aprendizaje que permite a los alumnos simular, probar e iterar sobre distintas configuraciones de EDPs — variando parámetros físicos, condiciones iniciales y dominios espaciales — para construir intuición sobre su comportamiento de forma activa. Como funcionalidad complementaria, el sistema incorpora un mecanismo de sincronización que permite al profesor propagar una configuración específica a todos los dispositivos conectados, facilitando la discusión guiada en clase a partir de un caso común.

El proyecto se estructura en dos etapas diferenciadas. La primera corresponde al **servidor**, implementado en el lenguaje **Go** utilizando el framework **Gin** para la gestión de rutas HTTP. El servidor expone una API REST que permite el registro de usuarios, el cálculo de soluciones numéricas y el control de sincronización entre dispositivos. Se implementaron dos solvers de ecuaciones diferenciales parciales: la **ecuación de onda 2D**, resuelta mediante el esquema explícito *leapfrog* de segundo orden con condición CFL, y la **ecuación de calor 2D** sobre una placa cuadrada, resuelta mediante el esquema *FTCS* (*Forward Time Centered Space*) explícito. Ambos solvers operan sobre un dominio espacial discretizado con condiciones de contorno de Dirichlet, y emplean técnicas de *rolling buffer* y submuestreo temporal para mantener el uso de memoria acotado independientemente de la duración de la simulación. El servidor incluye además un panel web docente desarrollado en HTML, CSS y JavaScript, que permite visualizar las soluciones en una representación tridimensional renderizada por software mediante un rasterizador de polígonos implementado sobre el elemento **Canvas** de HTML5, con controles de órbita y un sistema de sincronización de parámetros hacia los alumnos.

La segunda etapa corresponde a la **aplicación móvil de realidad virtual**, desarrollada en **Unity**. La aplicación consume la API del servidor y presenta las soluciones numéricas en un entorno de realidad virtual inmersivo, permitiendo al alumno interactuar con la simulación en tiempo real. La visualización en realidad virtual aporta una dimensión perceptual adicional respecto a las representaciones convencionales, facilitando la comprensión intuitiva del comportamiento de las EDPs en el espacio y el tiempo.

1. Introducción

Las ecuaciones diferenciales parciales (EDPs) constituyen una de las herramientas matemáticas fundamentales en ingeniería y ciencias aplicadas. Modelan fenómenos físicos tan diversos como la propagación de ondas, la difusión de calor, el flujo de fluidos y la deformación de estructuras. Sin embargo, su comprensión intuitiva representa un desafío significativo para los estudiantes, en parte debido a que los métodos de resolución analítica se limitan a casos muy particulares y las soluciones numéricas suelen presentarse de forma estática en hojas de cálculo o gráficas bidimensionales.

El presente proyecto propone un enfoque alternativo: una plataforma interactiva que permite al usuario simular el comportamiento de EDPs en tiempo real, variar libremente sus parámetros y condiciones iniciales, y observar las soluciones mediante visualizaciones tridimensionales inmersivas en realidad virtual. El objetivo no es reemplazar la teoría sino complementarla, ofreciendo un entorno donde la experimentación directa genere intuición física sobre fenómenos que de otro modo resultan abstractos.

El sistema se compone de dos componentes principales que se comunican mediante una arquitectura cliente-servidor HTTP. El servidor, desarrollado en Go, se encarga del cálculo numérico de las soluciones y de gestionar múltiples usuarios simultáneos con sus respectivas sesiones. El cliente, una aplicación móvil de realidad virtual desarrollada en Unity, consume las soluciones del servidor y las presenta al usuario en un entorno inmersivo e interactivo.

Como funcionalidad adicional orientada al uso en aula, el sistema incorpora un mecanismo de sincronización que permite al profesor propagar una configuración específica a todos los dispositivos conectados, habilitando la discusión guiada sobre un caso común.

2. Objetivos y Beneficiarios

2.1. Objetivo general

Desarrollar una plataforma cliente-servidor para la simulación interactiva de ecuaciones diferenciales parciales, con visualización en realidad virtual, que facilite el aprendizaje activo mediante la exploración paramétrica de soluciones numéricas en tiempo real.

2.2. Objetivos específicos

- Implementar solvers numéricos estables y eficientes para la ecuación de onda 2D y la ecuación de calor 2D sobre dominios cuadrados.
- Diseñar una API HTTP que exponga los resultados de la simulación a clientes heterogéneos (app móvil, panel web) de forma eficiente y con control de acceso por roles.
- Desarrollar una aplicación móvil en Unity que presente las soluciones numéricas en un entorno de realidad virtual inmersivo, permitiendo al usuario interactuar con la simulación.
- Proveer un panel web docente que permita visualizar, controlar y sincronizar simulaciones hacia múltiples usuarios conectados.
- Garantizar la estabilidad numérica de los esquemas implementados mediante el cumplimiento de las condiciones de estabilidad correspondientes a cada método.

2.3. Beneficiarios

La plataforma está orientada principalmente a **estudiantes de carreras de ingeniería y ciencias exactas** que cursen asignaturas donde las EDPs tienen un rol central: mecánica de medios continuos, transferencia de calor, electromagnetismo, mecánica de fluidos, entre otras. La posibilidad de explorar libremente parámetros y condiciones iniciales convierte al sistema en una herramienta de estudio autónomo que complementa la clase magistral.

En segundo lugar, la plataforma beneficia a los **docentes**, que pueden utilizarla como recurso didáctico durante la clase para ilustrar conceptos con simulaciones en vivo, o proponer a los alumnos escenarios específicos mediante la funcionalidad de sincronización.

3. Resolución de Ecuaciones Diferenciales

3.1. Fundamentos del método de diferencias finitas

Las EDPs implementadas en este proyecto se resuelven mediante el **método de diferencias finitas** (MDF), que consiste en discretizar el dominio continuo en una grilla de puntos y aproximar las derivadas parciales mediante cocientes de diferencias entre valores en nodos vecinos.

Dado un dominio espacial cuadrado $[0, L] \times [0, L]$ con n subdivisiones, el paso espacial es $\Delta x = \Delta y = L/n$. Los nodos interiores son los puntos $(i\Delta x, j\Delta y)$ con $1 \leq i, j \leq n-2$. Las condiciones de contorno de Dirichlet se imponen directamente sobre los nodos del borde, fijando su valor para todo tiempo.

La aproximación por diferencias centradas de segundo orden para la derivada segunda espacial en 2D es el **laplaciano discreto**:

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2} \quad (1)$$

Esta expresión es la base de ambos esquemas numéricos implementados.

3.2. Ecuación de onda 2D

3.2.1. Formulación matemática

La ecuación de onda 2D describe la evolución temporal de una membrana vibrante:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (2)$$

donde $u(x, y, t)$ es el desplazamiento de la membrana y c es la velocidad de propagación de la onda. Se imponen condiciones de contorno de Dirichlet homogéneas ($u = 0$ en todo el borde) y velocidad inicial nula ($\partial u / \partial t|_{t=0} = 0$).

3.2.2. Esquema leapfrog de segundo orden

Se utiliza el esquema explícito *leapfrog* centrado en tiempo y espacio, que aproxima la derivada segunda temporal:

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{u_{i,j}^{k+1} - 2u_{i,j}^k + u_{i,j}^{k-1}}{\Delta t^2} \quad (3)$$

Combinando con el laplaciano discreto (1) y despejando u^{k+1} :

$$u_{i,j}^{k+1} = 2u_{i,j}^k - u_{i,j}^{k-1} + r^2(u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k) \quad (4)$$

donde $r = c \Delta t / \Delta x$ es el **número de Courant**: mide cuántas celdas espaciales recorre la perturbación en un paso temporal. Un valor $r > 1/\sqrt{2}$ significa que la perturbación “salta” más rápido de lo que el esquema puede representar, provocando inestabilidad numérica.

Para el primer paso temporal, donde no existe u^{k-1} , se aplica la condición de velocidad inicial nula para obtener un arranque de primer orden:

$$u_{i,j}^1 = u_{i,j}^0 + \frac{r^2}{2} (u_{i+1,j}^0 + u_{i-1,j}^0 + u_{i,j+1}^0 + u_{i,j-1}^0 - 4u_{i,j}^0) \quad (5)$$

3.2.3. Condición de estabilidad CFL

Para que el esquema leapfrog sea estable en 2D, el número de Courant debe satisfacer:

$$r = \frac{c \Delta t}{\Delta x} \leq \frac{1}{\sqrt{2}} \quad (6)$$

En la implementación se usa $r = 0,9/\sqrt{2}$ para mantener un margen de seguridad. El paso temporal resultante es $\Delta t = r \Delta x / c$, y el número de pasos temporales necesario para simular T segundos es $n_t = \lceil T / \Delta t \rceil$.

3.2.4. Condiciones iniciales implementadas

Se ofrecen tres configuraciones de condición inicial $u(x, y, 0)$:

- **Seno**: modo fundamental de vibración de la membrana cuadrada:

$$u_0 = \sin\left(\frac{\pi x}{L}\right) \sin\left(\frac{\pi y}{L}\right)$$

- **Gaussiana**: pulso centrado que se propaga como ondas circulares:

$$u_0 = \exp\left[-\left(\frac{x - L/2}{L/5}\right)^2 - \left(\frac{y - L/2}{L/5}\right)^2\right]$$

- **Triangular**: pirámide lineal centrada:

$$u_0 = (1 - |2x/L - 1|)(1 - |2y/L - 1|)$$

3.2.5. Implementación en Go

```
1 func ecuacion_onda_2d(L, T, c float64, inicial string) [][][]float64 {
2     n := int(math.Ceil(L * 8))
3     if n < 16 { n = 16 }
4     d := L / float64(n)
5
6     CFL := 0.9 / math.Sqrt2
7     d_t := CFL * d / c
8     n_t := int(math.Ceil(T / d_t))
9     d_t = T / float64(n_t)
10    r := (c * d_t / d) * (c * d_t / d)
11
12    maxFrames := 200
13    step := 1
14    if n_t > maxFrames { step = n_t / maxFrames }
15
16    out := make([][][]float64, 0, maxFrames)
17    pp := newGrid(n) // t-2
```

```
18  p    := newGrid(n)    // t-1
19  cur  := newGrid(n)    // t
20
21  // Condicion inicial
22  for i := 1; i < n-1; i++ {
23      for j := 1; j < n-1; j++ {
24          x := float64(i) * d
25          y := float64(j) * d
26          switch inicial {
27              case "gauss":
28                  pp[i][j] = math.Exp(-math.Pow((x-L/2)/(L/5), 2) -
29                                     math.Pow((y-L/2)/(L/5), 2))
30              case "triangular":
31                  tx := 1 - math.Abs(2*x/L-1)
32                  ty := 1 - math.Abs(2*y/L-1)
33                  pp[i][j] = tx * ty
34              default: // seno
35                  pp[i][j] = math.Sin(math.Pi*x/L) * math.Sin(math.Pi*y/L)
36          }
37      }
38  }
39  out = append(out, copyGrid(pp))
40
41  // Primer paso (velocidad inicial = 0)
42  for i := 1; i < n-1; i++ {
43      for j := 1; j < n-1; j++ {
44          p[i][j] = pp[i][j] + 0.5*r*(pp[i+1][j]+pp[i-1][j]+
45                                     pp[i][j+1]+pp[i][j-1]-4*pp[i][j])
46      }
47  }
48  if step == 1 { out = append(out, copyGrid(p)) }
49
50  // Pasos restantes - leapfrog
51  for t := 2; t < n_t; t++ {
52      for i := 1; i < n-1; i++ {
53          for j := 1; j < n-1; j++ {
54              cur[i][j] = 2*p[i][j] - pp[i][j] +
55                          r*(p[i+1][j]+p[i-1][j]+p[i][j+1]+p[i][j-1]-4*p[i][j])
56          }
57      }
58      if t%step == 0 { out = append(out, copyGrid(cur)) }
59      pp, p, cur = p, cur, pp
60  }
61  return out
62 }
```

Listing 1: Solver de la ecuación de onda 2D — solver.go

3.3. Ecuación de calor 2D

3.3.1. Formulación matemática

La ecuación de calor 2D describe la difusión de temperatura sobre una placa cuadrada:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (7)$$

donde $u(x, y, t)$ es la temperatura normalizada y α es la difusividad térmica del material.

3.3.2. Esquema FTCS explícito

Se utiliza el esquema *Forward Time Centered Space* (FTCS), que aproxima la derivada temporal mediante diferencias hacia adelante de primer orden:

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j}^{k+1} - u_{i,j}^k}{\Delta t} \quad (8)$$

Combinando con el laplaciano discreto (1) y despejando u^{k+1} :

$$u_{i,j}^{k+1} = u_{i,j}^k + r_\alpha (u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k) \quad (9)$$

donde $r_\alpha = \alpha \Delta t / \Delta x^2$ es el número de difusión.

A diferencia del esquema leapfrog, el FTCS solo requiere el instante anterior para avanzar en tiempo, lo que simplifica la implementación al necesitar únicamente dos grillas en lugar de tres.

3.3.3. Condición de estabilidad

Para que el esquema FTCS sea estable en 2D, el número de difusión debe satisfacer:

$$r_\alpha = \frac{\alpha \Delta t}{\Delta x^2} \leq \frac{1}{4} \quad (10)$$

En la implementación se usa $r_\alpha = 0,20$ como margen de seguridad respecto al límite teórico de 0,25.

3.3.4. Condiciones iniciales y de contorno implementadas

Se ofrecen dos configuraciones:

- **Gaussiana:** pulso de temperatura centrado. Los cuatro bordes se fijan en $u = 0$ (sumideros fríos). El calor se disipa hacia los bordes y la temperatura interior decae hasta equilibrarse en cero.
- **Bordes calientes:** los cuatro bordes se fijan en $u = 1$ (fuentes de calor constantes) y el interior comienza en $u = 0$. El calor penetra desde las paredes hacia el centro hasta alcanzar el estado estacionario $u = 1$ en todo el dominio.

En ambos casos las condiciones de contorno se reimponen explícitamente después de cada paso temporal para mantener el valor fijo en el borde independientemente de la difusión numérica.

3.3.5. Implementación en Go

```
1 func ecuacion_calor_2d(L, T, alpha float64, inicial string) [][][]  
   float64 {  
2     n := int(math.Ceil(L * 5))  
3     if n < 10 { n = 10 }  
4     d := L / float64(n)  
5  
6     r_max := 0.20  
7     d_t := r_max * d * d / alpha  
8     n_t := int(math.Ceil(T / d_t))  
9     d_t = T / float64(n_t)  
10    r := alpha * d_t / (d * d)  
11  
12    maxFrames := 200  
13    step := 1  
14    if n_t > maxFrames { step = n_t / maxFrames }  
15  
16    out := make([][][]float64, 0, maxFrames)  
17    prev := newGrid(n)  
18    curr := newGrid(n)  
19  
20    bcVal := 0.0  
21    if inicial == "bordes" { bcVal = 1.0 }  
22  
23    // Condicion inicial  
24    for i := 1; i < n-1; i++ {  
25        for j := 1; j < n-1; j++ {  
26            x := float64(i) * d  
27            y := float64(j) * d  
28            switch inicial {  
29                case "gauss":  
30                    prev[i][j] = math.Exp(-(math.Pow((x-L/2)/(L/5), 2) +  
31                                            math.Pow((y-L/2)/(L/5), 2)))  
32                    default: // bordes calientes: interior frio  
33                        prev[i][j] = 0.0  
34                }  
35            }  
36        }  
37    // Aplicar condicion de contorno inicial  
38    for k := 0; k < n; k++ {  
39        prev[k][0] = bcVal; prev[k][n-1] = bcVal  
40        prev[0][k] = bcVal; prev[n-1][k] = bcVal  
41    }  
42    out = append(out, copyGrid(prev))  
43  
44    // Avance temporal FTCS  
45    for t := 1; t < n_t; t++ {  
46        for i := 1; i < n-1; i++ {  
47            for j := 1; j < n-1; j++ {  
48                curr[i][j] = prev[i][j] + r*(prev[i+1][j]+prev[i-1][j]+  
49                    prev[i][j+1]+prev[i][j-1]-4*prev[i][j])  
50            }  
51        }  
52    // Reimposicion de condicion de contorno  
53    for k := 0; k < n; k++ {
```

```
54         curr[k][0] = bcVal; curr[k][n-1] = bcVal
55         curr[0][k] = bcVal; curr[n-1][k] = bcVal
56     }
57     if t%step == 0 { out = append(out, copyGrid(curr)) }
58     prev, curr = curr, prev
59 }
60 return out
61 }
```

Listing 2: Solver de la ecuación de calor 2D — `solver.go`

3.4. Consideraciones de eficiencia

Ambos solvers comparten dos estrategias para mantener el uso de recursos acotado independientemente de la duración o resolución de la simulación:

- **Rolling buffer:** en lugar de almacenar todos los pasos temporales en memoria, se mantienen únicamente dos o tres grillas activas que se rotan mediante reasignación de punteros (`pp, p, cur = p, cur, pp`). El costo de memoria es $O(n^2)$ constante.
- **Submuestreo temporal:** si la simulación requiere más de 200 pasos temporales, solo se guarda uno de cada $\lfloor n_t/200 \rfloor$ pasos. Esto limita el tamaño de la respuesta JSON a un máximo de 200 frames sin afectar la estabilidad numérica, que depende del paso Δt interno y no de cuántos frames se exportan.

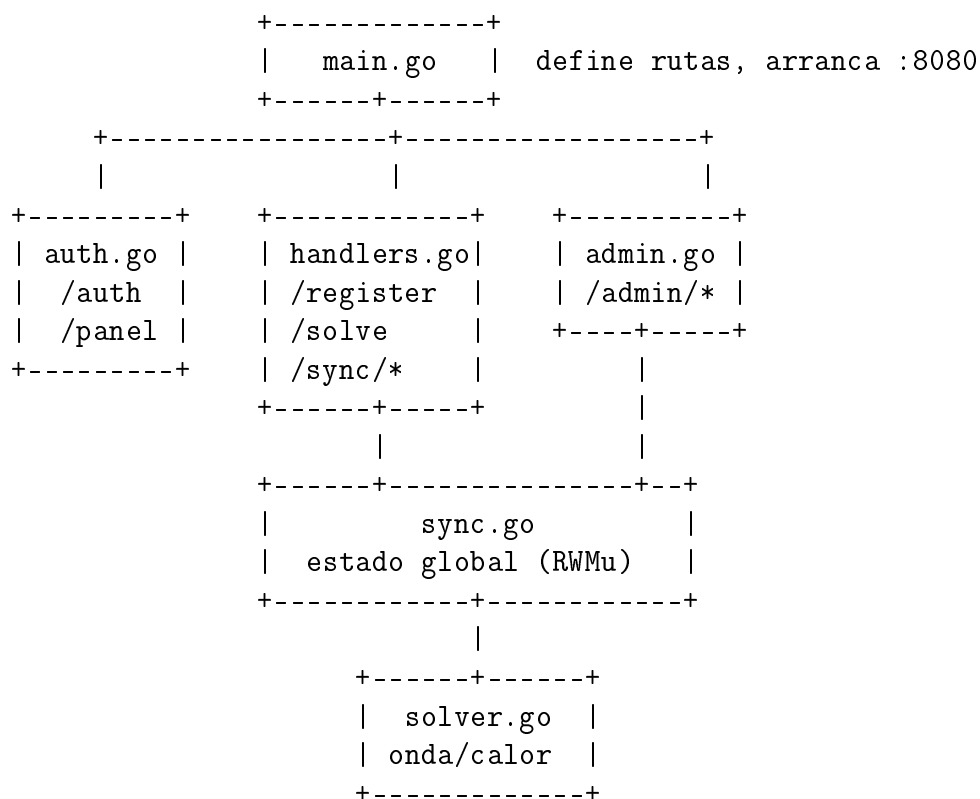
4. Arquitectura del Servidor

El servidor está escrito íntegramente en Go y utiliza el framework **Gin** para la gestión de rutas HTTP. El código se organiza en cinco archivos, cada uno con una responsabilidad bien delimitada:

Archivo	Responsabilidad
<code>main.go</code>	Punto de entrada: router y arranque del servidor
<code>auth.go</code>	Autenticación del profesor y limitación de tasa
<code>sync.go</code>	Estado global en memoria: clientes, roles y sincronización
<code>handlers.go</code>	Endpoints de la app móvil
<code>admin.go</code>	Endpoints del panel web docente
<code>solver.go</code>	Solvers numéricos de EDPs (sin dependencias HTTP)

Todos los archivos pertenecen al mismo paquete `main`, lo que significa que comparten tipos, variables y funciones sin necesidad de importaciones entre ellos. El estado global del servidor vive en una única variable (`estado`) definida en `sync.go`, y todos los handlers acceden a ella mediante sus métodos.

El diagrama siguiente resume cómo se conectan los archivos:



4.1. Resumen de rutas

La tabla siguiente sintetiza todos los endpoints expuestos por el servidor, el método HTTP, quién puede usarlos y qué devuelven.

Ruta	Método	Acceso	Función / Respuesta
/	GET	Todos	Sirve la pantalla de login del profesor (<code>login.html</code>)
/auth	POST	Todos	Verifica el código del profesor y autoriza su IP. Devuelve <code>{ok:true}</code>
/panel	GET	Profesor (IP autorizada)	Sirve el panel docente (<code>panel.html</code>). Redirige al login si la IP no está autorizada
/register	POST	App móvil	Registra o actualiza un alumno (legajo, nombre). Devuelve <code>{ok:true}</code>
/solve	GET	App móvil (1 req/5s)	Calcula la solución de la EDP con los parámetros dados. Devuelve <code>{sync, ecuacion, result}</code> con hasta 200 frames
/sync/set	POST	Asistente	Activa la sincronización con los parámetros indicados. Solo accesible con rol de asistente
/sync/clear	POST	Asistente	Desactiva la sincronización. Solo accesible con rol de asistente
/admin/clients	GET	Profesor	Lista todos los alumnos registrados con IP, legajo, nombre, rol y hora del último request
/admin/promote	POST	Profesor	Otorga rol de asistente al cliente con la IP indicada
/admin/demote	POST	Profesor	Revoca el rol de asistente del cliente con la IP indicada
/admin/sync/set	POST	Profesor	Activa la sincronización directamente desde el panel, sin requerir asistente
/admin/sync/clear	POST	Profesor	Desactiva la sincronización desde el panel

4.2. main.go — Router y punto de entrada

main.go es el punto de entrada del servidor. Declara la constante del código de acceso del profesor y en main() crea el router, registra todas las rutas y arranca el servidor en el puerto 8080.

```
1 package main
2
3 import "github.com/gin-gonic/gin"
4
5 const codigoProfesor = "123"
6
7 func main() {
8     router := gin.Default()
```

Listing 3: main.go completo

gin.Default() crea un router con los middlewares de registro de requests y recuperación de panics ya activados. El código del profesor está hardcodeado por simplicidad: en el contexto de aula esto es aceptable, ya que la red es controlada y el código se cambia directamente en el binario si es necesario.

```
1     router.StaticFile("/", "./web/login.html")
2     router.POST("/auth", authHandler)
3     router.GET("/panel", panelHandler)
```

Estas tres rutas forman el flujo de autenticación del profesor. La raíz sirve la pantalla de login. POST /auth valida el código y registra la IP. GET /panel sirve el panel solo si la IP ya fue autorizada.

```
1     admin := router.Group("/admin")
2     {
3         admin.GET("/clients", clientesHandler)
4         admin.POST("/promote", promoverHandler)
5         admin.POST("/demote", demoverHandler)
6         admin.POST("/sync/clear", syncClearHandler)
7         admin.POST("/sync/set", syncSetAdminHandler)
8     }
```

El grupo /admin agrupa todas las rutas exclusivas del panel docente bajo un prefijo común. Si en el futuro se quisiera añadir un middleware de verificación de IP a todas ellas, bastaría con agregarlo al grupo sin tocar cada handler.

```
1     router.POST("/register", registerHandler)
2     router.GET("/solve", RateLimitMiddleware(), solveHandler)
3     router.POST("/sync/set", syncSetHandler)
4     router.POST("/sync/clear", syncClearAsistenteHandler)
5
6     router.Run("10.83.158.17:8080")
7 }
```

Las rutas de la app móvil son públicas: cualquier dispositivo en la red puede acceder a ellas. La excepción es /solve, que pasa primero por RateLimitMiddleware(): este middleware limita a una solicitud cada cinco segundos por IP, evitando que un alumno sature el servidor con cálculos costosos consecutivos.

4.3. auth.go — Autenticación y limitación de tasa

Este archivo implementa dos mecanismos de control de acceso independientes: la autenticación del profesor por IP y el limitador de tasa para el endpoint de cálculo.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6     "net/http"
7     "sync"
8     "time"
9
10    "github.com/gin-gonic/gin"
11    "golang.org/x/time/rate"
12 )
13
14 type authRequest struct {
15     Codigo string `json:"codigo"`
16 }
17
18 var (
19     ipsAutorizadas = make(map[string]bool)
20     ipsAutorizadasMu sync.RWMutex
21 )
22
23 var limiters sync.Map
```

Listing 4: auth.go — imports y variables globales

`ipsAutorizadas` es el mapa que actúa como lista blanca de IPs del profesor. Está protegido por un `sync.RWMutex` propio (distinto del mutex del estado global) porque es independiente del ciclo de vida de los clientes. `limiters` es un `sync.Map` — un mapa diseñado para acceso concurrente sin necesidad de mutex manual — que almacena un limitador de tasa por IP.

```
1 func getLimiter(ip string) *rate.Limiter {
2     if v, ok := limiters.Load(ip); ok {
3         return v.(*rate.Limiter)
4     }
5     lim := rate.NewLimiter(rate.Every(5*time.Second), 1)
6     actual, _ := limiters.LoadOrStore(ip, lim)
7     return actual.(*rate.Limiter)
8 }
9
10 func RateLimitMiddleware() gin.HandlerFunc {
11     return func(c *gin.Context) {
12         ip := c.ClientIP()
13         lim := getLimiter(ip)
14
15         res := lim.Reserve()
16         if !res.OK() {
17             c.AbortWithStatusJSON(http.StatusTooManyRequests,
18                 gin.H{"error": "Demasiadas peticiones"})
19             return
20         }
21     }
22 }
```

```
20     }
21
22     delay := res.Delay()
23     if delay <= 0 {
24         c.Next()
25         return
26     }
27
28     res.Cancel()
29     sec := int(math.Ceil(delay.Seconds()))
30     c.Header("Retry-After", fmt.Sprintf("%d", sec))
31     c.AbortWithStatusJSON(http.StatusTooManyRequests, gin.H{
32         "error":      "Demasiadas peticiones",
33         "waitSeconds": sec,
34         "message":     fmt.Sprintf("Espera %ds antes de reintentar",
35                                 sec),
36     })
37 }
```

Listing 5: Limitador de tasa por IP

`rate.NewLimiter(rate.Every(5*time.Second), 1)` crea un limitador de tipo *token bucket* con capacidad de un token que se regenera cada cinco segundos. `Reserve()` consume el token si está disponible y devuelve cuánto tiempo habría que esperar de lo contrario. Si el delay es mayor que cero, el middleware cancela la reserva (para no consumir el token inútilmente), agrega la cabecera `Retry-After` y rechaza el request con código 429.

```
1 func autorizarIP(ip string) {
2     ipsAutorizadasMu.Lock()
3     defer ipsAutorizadasMu.Unlock()
4     ipsAutorizadas[ip] = true
5     fmt.Println("[auth] IP autorizada:", ip)
6 }
7
8 func ipAutorizada(ip string) bool {
9     ipsAutorizadasMu.RLock()
10    defer ipsAutorizadasMu.RUnlock()
11    return ipsAutorizadas[ip]
12 }
13
14 func authHandler(c *gin.Context) {
15     var req authRequest
16     if err := c.ShouldBindJSON(&req); err != nil {
17         c.JSON(http.StatusBadRequest,
18             gin.H{"error": "Cuerpo de solicitud invalido"})
19         return
20     }
21     if req.Codigo != codigoProfesor {
22         c.JSON(http.StatusUnauthorized, gin.H{"error": "Codigo
23             incorrecto"})
24         return
25     }
26     autorizarIP(c.ClientIP())
27     c.JSON(http.StatusOK, gin.H{"ok": true})
28 }
```



```
28
29 func panelHandler(c *gin.Context) {
30     if !ipAutorizada(c.ClientIP()) {
31         c.Redirect(http.StatusFound, "/")
32         return
33     }
34     c.File("./web/panel.html")
35 }
```

Listing 6: Autenticación del profesor

`authHandler` valida el código y delega en `autorizarIP`, que escribe en el mapa con lock exclusivo. `panelHandler` usa `RLock` (lectura compartida) para verificar la IP, ya que múltiples profesores podrían abrir el panel simultáneamente sin interferirse. Si la IP no está autorizada, redirige al login en lugar de devolver un error, para que el flujo sea transparente en el navegador.

4.4. sync.go — Estado global en memoria

Este archivo es el núcleo del servidor: define todos los tipos de datos y centraliza el estado compartido entre todos los handlers mediante una única variable global `estado`.

```
1 package main
2
3 import (
4     "sync"
5     "time"
6 )
7
8 type rol int
9
10 const (
11     rolAlumno    rol = iota
12     rolAsistente rol = iota
13 )
14
15 type Cliente struct {
16     IP          string
17     Legajo      string
18     Nombre     string
19     Rol        rol
20     UltimoRequest time.Time
21 }
22
23 type parametrosSync struct {
24     Ecuacion string
25     L        float64
26     T        float64
27     C        float64
28     Inicial  string
29 }
30
31 type estadoGlobal struct {
32     mu      sync.RWMutex
33     clientes map[string]*Cliente
```

```
34     sync      struct {
35         activo    bool
36         parametros parametrosSync
37     }
38 }
39
40 var estado = &estadoGlobal{
41     clientes: make(map[string]*Cliente),
42 }
```

Listing 7: sync.go — tipos y estado global

rol usa el patrón *iota* de Go para definir constantes enteras enumeradas. `estadoGlobal` agrupa clientes y estado de sincronización bajo un único `sync.RWMutex`: esto garantiza que todas las operaciones sobre el estado sean atómicas y que múltiples goroutines (una por request) puedan leer simultáneamente sin bloquearse entre sí, reservando la exclusividad solo para escrituras.

```
1 func (e *estadoGlobal) RegistrarCliente(ip, legajo, nombre string) {
2     e.mu.Lock()
3     defer e.mu.Unlock()
4
5     if c, existe := e.clientes[ip]; existe {
6         c.Legajo = legajo
7         c.Nombre = nombre
8         c.UltimoRequest = time.Now()
9         return
10    }
11
12    e.clientes[ip] = &Cliente{
13        IP:            ip,
14        Legajo:        legajo,
15        Nombre:        nombre,
16        Rol:           rolAlumno,
17        UltimoRequest: time.Now(),
18    }
19 }
20
21 func (e *estadoGlobal) ActualizarRequest(ip string) {
22     e.mu.Lock()
23     defer e.mu.Unlock()
24     if c, existe := e.clientes[ip]; existe {
25         c.UltimoRequest = time.Now()
26     }
27 }
```

Listing 8: Registro de clientes

`RegistrarCliente` es idempotente: si la IP ya existe, actualiza legajo y nombre pero *no* toca el campo `Rol`. Esto es fundamental para que un asistente pueda cerrar y reabrir la app sin perder el rol que le otorgó el profesor.

```
1 func (e *estadoGlobal) Clientes() []*Cliente {
2     e.mu.RLock()
3     defer e.mu.RUnlock()
4     lista := make([]*Cliente, 0, len(e.clientes))
```

```
5     for _, c := range e.clientes {
6         lista = append(lista, c)
7     }
8     return lista
9 }
10
11 func (e *estadoGlobal) BuscarCliente(ip string) *Cliente {
12     e.mu.RLock()
13     defer e.mu.RUnlock()
14     return e.clientes[ip]
15 }
16
17 func (e *estadoGlobal) PromoverAsistente(ip string) bool {
18     e.mu.Lock()
19     defer e.mu.Unlock()
20     c, existe := e.clientes[ip]
21     if !existe { return false }
22     c.Rol = rolAsistente
23     return true
24 }
25
26 func (e *estadoGlobal) DemoverAsistente(ip string) bool {
27     e.mu.Lock()
28     defer e.mu.Unlock()
29     c, existe := e.clientes[ip]
30     if !existe { return false }
31     c.Rol = rolAlumno
32     return true
33 }
```

Listing 9: Consulta y modificacion de clientes

Las operaciones de lectura usan RLock/RUnlock (múltiples lectores simultáneos); las de escritura usan Lock/Unlock (exclusividad total). defer garantiza que el mutex se libere siempre al salir de la función, incluso si ocurre un panic.

```
1 func (e *estadoGlobal) ActivarSync(p parametrosSync) {
2     e.mu.Lock()
3     defer e.mu.Unlock()
4     e.sync.activo = true
5     e.sync.parametros = p
6 }
7
8 func (e *estadoGlobal) DesactivarSync() {
9     e.mu.Lock()
10    defer e.mu.Unlock()
11    e.sync.activo = false
12    e.sync.parametros = parametrosSync{}
13 }
14
15 func (e *estadoGlobal) EstadoSync() (bool, parametrosSync) {
16     e.mu.RLock()
17     defer e.mu.RUnlock()
18     return e.sync.activo, e.sync.parametros
19 }
```

Listing 10: Control de sincronizacion

El mecanismo de sincronización es un canal de difusión pasivo: el asistente o el profesor escriben los parámetros una vez, y todos los alumnos los reciben la próxima vez que llaman a `/solve`. No hay notificaciones push ni websockets; la app consulta periódicamente y detecta el cambio por el campo `"sync"` de la respuesta.

4.5. `handlers.go` — Endpoints de la app móvil

Este archivo implementa los cuatro endpoints con los que interactúa la aplicación Unity: registro, cálculo y control de sincronización desde el rol de asistente.

```
1 package main
2
3 import (
4     "net/http"
5     "strconv"
6     "github.com/gin-gonic/gin"
7 )
8
9 type registerRequest struct {
10     Legajo string `json:"legajo"`
11     Nombre string `json:"nombre"`
12 }
13
14 func registerHandler(c *gin.Context) {
15     var req registerRequest
16     if err := c.ShouldBindJSON(&req); err != nil {
17         c.JSON(http.StatusBadRequest,
18             gin.H{"error": "Cuerpo de solicitud invalido"})
19         return
20     }
21     if req.Legajo == "" || req.Nombre == "" {
22         c.JSON(http.StatusBadRequest,
23             gin.H{"error": "Legajo y nombre son obligatorios"})
24         return
25     }
26     estado.RegistrarCliente(c.ClientIP(), req.Legajo, req.Nombre)
27     c.JSON(http.StatusOK, gin.H{"ok": true})
28 }
```

Listing 11: `handlers.go` — registro de alumno

`ShouldBindJSON` deserializa el cuerpo de la petición en el struct e informa el error si el JSON es malformado. La validación de campos vacíos es explícita porque `ShouldBindJSON` no valida semántica, solo sintaxis. La IP del cliente la provee Gin directamente con `c.ClientIP()`.

```
1 func solveHandler(c *gin.Context) {
2     estado.ActualizarRequest(c.ClientIP())
3
4     syncActivo, params := estado.EstadoSync()
5
6     var L, T, cw float64
7     var inicial, ecuacion string
8
9     if syncActivo {
```

```
10     L      = params.L
11     T      = params.T
12     cw     = params.C
13     inicial = params.Inicial
14     ecuacion = params.Ecuacion
15 } else {
16     Ls, ok1 := c.GetQuery("L")
17     Ts, ok2 := c.GetQuery("T")
18     cs, ok3 := c.GetQuery("c")
19     inicial, _ = c.GetQuery("inicial")
20     ecuacion, _ = c.GetQuery("ecuacion")
21
22     if !ok1 || !ok2 || !ok3 {
23         c.JSON(http.StatusBadRequest,
24             gin.H{"error": "Faltan parametros en la consulta"})
25         return
26     }
27
28     var err1, err2, err3 error
29     L, err1 = strconv.ParseFloat(Ls, 64)
30     T, err2 = strconv.ParseFloat(Ts, 64)
31     cw, err3 = strconv.ParseFloat(cs, 64)
32
33     if err1 != nil || err2 != nil || err3 != nil {
34         c.JSON(http.StatusBadRequest, gin.H{"error": "Parametros
35             invalidos"})
36         return
37     }
38
39     lMax := 10.0
40     if ecuacion == "calor2d" { lMax = 20.0 }
41     if L < 1 || L > lMax || T < 1 || T > 60 || cw < 0.1 || cw > 2 {
42         c.JSON(http.StatusBadRequest,
43             gin.H{"error": "Parametros fuera de rango"})
44         return
45     }
46
47     var result interface{}
48     switch ecuacion {
49     case "calor2d":
50         result = ecuacion_calor_2d(L, T, cw, inicial)
51     default:
52         ecuacion = "onda2d"
53         result = ecuacion_onda_2d(L, T, cw, inicial)
54     }
55
56     c.JSON(http.StatusOK, gin.H{
57         "sync":      syncActivo,
58         "ecuacion":  ecuacion,
59         "result":    result,
60     })
61 }
```

Listing 12: handlers.go — calculo de la EDP

Este handler implementa la lógica central del sistema. Lo primero que hace es consultar el estado de sincronización. Si está activo, ignora completamente los query parameters de la URL y usa los parámetros del asistente. Si no, los lee, los convierte de string a float64 (`strconv.ParseFloat`) y valida rangos. El límite de L varía según la ecuación: 10 m para onda 2D (donde la grilla crece como $n = 8L$) y 20 m para calor 2D (donde crece como $n = 5L$), controlando el tamaño máximo de la grilla.

```
1 type syncSetRequest struct {
2     Ecuacion string 'json:"ecuacion"'
3     L         float64 'json:"L"'
4     T         float64 'json:"T"'
5     C         float64 'json:"c"'
6     Inicial  string 'json:"inicial"'
7 }
8
9 func syncSetHandler(c *gin.Context) {
10     cliente := estado.BuscarCliente(c.ClientIP())
11     if cliente == nil || cliente.Rol != rolAsistente {
12         c.JSON(http.StatusForbidden, gin.H{"error": "Se requiere rol de
13             asistente"})
14         return
15     }
16     var req syncSetRequest
17     if err := c.ShouldBindJSON(&req); err != nil {
18         c.JSON(http.StatusBadRequest,
19             gin.H{"error": "Cuerpo de solicitud invalido"})
20         return
21     }
22     lMax := 10.0
23     if req.Ecuacion == "calor2d" { lMax = 20.0 }
24     if req.L < 1 || req.L > lMax || req.T < 1 || req.T > 60 ||
25         req.C < 0.1 || req.C > 2 {
26         c.JSON(http.StatusBadRequest, gin.H{"error": "Parametros fuera
27             de rango"})
28         return
29     }
30     estado.ActivarSync(parametrosSync{
31         Ecuacion: req.Ecuacion,
32         L: req.L, T: req.T, C: req.C, Inicial: req.Inicial,
33     })
34     c.JSON(http.StatusOK, gin.H{"ok": true})
35 }
36
37 func syncClearAsistenteHandler(c *gin.Context) {
38     cliente := estado.BuscarCliente(c.ClientIP())
39     if cliente == nil || cliente.Rol != rolAsistente {
40         c.JSON(http.StatusForbidden, gin.H{"error": "Se requiere rol de
41             asistente"})
42         return
43     }
44     estado.DesactivarSync()
45     c.JSON(http.StatusOK, gin.H{"ok": true})
46 }
```

Listing 13: handlers.go — sincronizacion desde asistente

Ambos handlers verifican el rol antes de cualquier otra operación. Si la IP no existe en el mapa de clientes (dispositivo no registrado) o su rol no es asistente, se rechaza con 403. La verificación de rol es la única diferencia con los endpoints equivalentes de `admin.go`.

4.6. `admin.go` — Endpoints del panel docente

Este archivo implementa las rutas del grupo `/admin`. Su lógica es análoga a `handlers.go` pero sin verificación de rol de asistente, ya que el acceso al grupo `/admin` implica que el profesor ya fue autenticado por IP.

```
1 package main
2
3 import (
4     "net/http"
5     "github.com/gin-gonic/gin"
6 )
7
8 type clienteResponse struct {
9     IP          string `json:"ip"`
10    Legajo       string `json:"legajo"`
11    Nombre       string `json:"nombre"`
12    Rol          string `json:"rol"`
13    UltimoRequest string `json:"ultimoRequest"`
14 }
15
16 func clientesHandler(c *gin.Context) {
17     clientes := estado.Clientes()
18     lista := make([]clienteResponse, 0, len(clientes))
19     for _, cl := range clientes {
20         rolStr := "alumno"
21         if cl.Rol == rolAsistente { rolStr = "asistente" }
22         lista = append(lista, clienteResponse{
23             IP:          cl.IP,
24             Legajo:       cl.Legajo,
25             Nombre:       cl.Nombre,
26             Rol:          rolStr,
27             UltimoRequest: cl.UltimoRequest.Format("15:04:05"),
28         })
29     }
30     c.JSON(http.StatusOK, gin.H{"clientes": lista})
31 }
```

Listing 14: `admin.go` — lista de clientes

Se define un struct `clienteResponse` separado del `Cliente` interno. Esta separación es deliberada: el struct interno usa `rol` (entero) y `time.Time`, pero la API expone strings legibles. Si el struct interno cambia, el contrato de la API no se ve afectado. `UltimoRequest.Format("15:04:05")` formatea el timestamp como hora en formato HH:MM:SS.

```
1 type promoverRequest struct {
2     IP string `json:"ip"`
3 }
4
5 func promoverHandler(c *gin.Context) {
```

```
6   var req promoverRequest
7   if err := c.ShouldBindJSON(&req); err != nil {
8       c.JSON(http.StatusBadRequest,
9           gin.H{"error": "Cuerpo de solicitud invalido"})
10      return
11  }
12  if !estado.PromoverAsistente(req.IP) {
13      c.JSON(http.StatusNotFound, gin.H{"error": "Cliente no
14          encontrado"})
15      return
16  }
17  c.JSON(http.StatusOK, gin.H{"ok": true})
18  }
19  func demoverHandler(c *gin.Context) {
20      var req promoverRequest
21      if err := c.ShouldBindJSON(&req); err != nil {
22          c.JSON(http.StatusBadRequest,
23              gin.H{"error": "Cuerpo de solicitud invalido"})
24          return
25      }
26      if !estado.DemoverAsistente(req.IP) {
27          c.JSON(http.StatusNotFound, gin.H{"error": "Cliente no
28              encontrado"})
29          return
30      }
31      c.JSON(http.StatusOK, gin.H{"ok": true})
32  }
33  func syncClearHandler(c *gin.Context) {
34      estado.DesactivarSync()
35      c.JSON(http.StatusOK, gin.H{"ok": true})
36  }
37
38  func syncSetAdminHandler(c *gin.Context) {
39      var req syncSetRequest
40      if err := c.ShouldBindJSON(&req); err != nil {
41          c.JSON(http.StatusBadRequest,
42              gin.H{"error": "Cuerpo de solicitud invalido"})
43          return
44      }
45      lMax := 10.0
46      if req.Ecuacion == "calor2d" { lMax = 20.0 }
47      if req.L < 1 || req.L > lMax || req.T < 1 || req.T > 60 ||
48          req.C < 0.1 || req.C > 2 {
49          c.JSON(http.StatusBadRequest, gin.H{"error": "Parametros fuera
50              de rango"})
51          return
52      }
53      estado.ActivarSync(parametrosSync{
54          Ecuacion: req.Ecuacion,
55          L: req.L, T: req.T, C: req.C, Inicial: req.Inicial,
56      })
57      c.JSON(http.StatusOK, gin.H{"ok": true})
58  }
```

Listing 15: `admin.go` — promote, demote y sync

`promoverHandler` y `demoverHandler` reutilizan el mismo struct `promoverRequest` porque ambas operaciones solo necesitan la IP del cliente a modificar. `syncClearHandler` no necesita validar nada: el profesor ya está autenticado y desactivar una sincronización inexistente es una operación inocua. `syncSetAdminHandler` es equivalente a `syncSetHandler` de la app móvil, con la diferencia de que no verifica rol de asistente.

5. Trabajo Futuro

El presente proyecto establece una base funcional para la simulación interactiva de EDPs en realidad virtual, pero abre también varias líneas de trabajo que exceden su alcance actual y que podrían ser abordadas en etapas posteriores.

5.1. Escalabilidad de la plataforma

La arquitectura actual está concebida para un entorno de aula con decenas de usuarios simultáneos conectados a una red local. Escalar la plataforma a un uso más amplio implicaría abordar al menos los siguientes aspectos:

- **Persistencia de sesiones:** el estado de los clientes se mantiene en memoria volátil del servidor. Incorporar una base de datos permitiría sobrevivir reinicios y registrar el historial de simulaciones de cada usuario.
- **Cómputo distribuido:** para dominios grandes o parámetros que requieran grillas de alta resolución, los solvers podrían distribuirse entre múltiples núcleos o nodos mediante goroutines paralelas o una cola de tareas, reduciendo los tiempos de respuesta bajo carga concurrente.
- **Despliegue en la nube:** migrar el servidor a una instancia en la nube eliminaría la restricción de red local, permitiendo que estudiantes accedan a la plataforma desde cualquier ubicación. Esto requeriría además incorporar un sistema de autenticación robusto en reemplazo del esquema de IP actual.
- **Limitación de recursos por usuario:** implementar colas de trabajo y límites de tamaño de grilla por sesión para evitar que simulaciones costosas de un usuario degraden la experiencia del resto.

5.2. Incorporación de nuevos solvers

La arquitectura modular del servidor facilita la incorporación de nuevas ecuaciones diferenciales. Algunas extensiones de interés pedagógico son:

- **Ecuación de Laplace / Poisson:** resolución de problemas electrostáticos y de potencial en estado estacionario mediante métodos iterativos (Jacobi, Gauss-Seidel, SOR). Permitiría visualizar distribuciones de campo eléctrico en dominios con geometrías arbitrarias.
- **Ecuación de onda 1D:** versión simplificada de la membrana vibrante, ideal como paso introductorio antes de los casos 2D. Su baja carga computacional permitiría resoluciones muy finas y tiempos de simulación largos.
- **Navier-Stokes simplificado:** simulación de flujo de fluidos incompresible en 2D (ecuaciones de corriente-vorticidad). Abre la plataforma a cursos de mecánica de fluidos y dinámica de fluidos computacional.
- **Esquemas implícitos:** para la ecuación de calor, un esquema de Crank-Nicolson o totalmente implícito eliminaría la restricción de estabilidad sobre Δt , permitiendo simular escalas temporales mucho más largas sin incrementar el cómputo por paso.

5.3. Mejoras en la experiencia de realidad virtual

Más allá de los solvers, la experiencia inmersiva podría enriquecerse con:

- Interacción gestual para modificar parámetros directamente dentro del entorno VR, sin salir de la aplicación.
- Representaciones volumétricas para problemas 3D, aprovechando la profundidad espacial que ofrece la realidad virtual.
- Modo colaborativo en tiempo real, donde múltiples usuarios en el entorno VR puedan modificar condiciones iniciales conjuntamente y observar el efecto de forma sincrónica.